

Guía de Creación de Driver De Mensajes:

En la librería Lantz se pueden crear drivers para dispositivos basados en mensajes. Estos pueden ser mensajes enviado por USB, por TCP/IP, o por puerto serial.

0. Software Requerido:

El software necesario para hacer funcionar estos generalmente es:

- Python 3
- Lantz (full)
- NI-VISA*

*Una copia de la documentación oficial de NI-VISA se encuentra en esta carpeta, en la subcarpeta llamada "NI-VISA Documentation"

1. Herencia:

Los Drivers que quieren conectarse a través de mensajes tendrían que utilizar de clase base "MessageBasedDriver", que ofrece una cantidad de métodos importantes e utiles para la conexión.

Cabecera de la clase:

```
class MessageBasedDriver(Driver):  
    """Base class for message based drivers using PyVISA as underlying library.  
  
    Notice that PyVISA can communicate using different backends. For example:  
    - @ni: Using NI-VISA for communication. Backend bundled with PyVISA.  
    - @py: Using PySerial, PyUSB and linux-gpib for communication. Available with PyVISA-py package.  
    - @sim: Simulated devices. Available with PyVISA-sim package.
```

Los métodos de clase "via_usb", "via_serial", "via_tcpip" y "via_gpib" son los métodos que permiten crear un driver con el tipo de conexión indicado. Estos métodos toman como parámetro generalmente la dirección donde se encuentra el dispositivo. TCP IP, por ejemplo, toma el IP y el puerto del dispositivo.

Ej. de uso:

```
driver = wrap_driver_cls(SR830).via_gpib(self.conf["CONNECTION"]["gpib_address"])  
driver.initialize()
```

El método para inicializar el driver es el segundo "initialize()" esto asegura que se conecte correctamente el driver al dispositivo y avisa si hay algún problema en la conexión.

2. Enviar Mensajes:

El envío de mensajes se hace a traves del método "write" presente en la clase. Este método escribe el mensaje en el canal de comunicación con el driver, y devuelve la cantidad de bytes que se escribieron. Esto se usaría en los casos donde es necesario solo escribir algo y no importa la respuesta.

Ejemplo de uso en el driver sgd5122:

```
@frequency.setter  
def frequency(self, channel, value):  
    """  
    Sets the frequency of the specified channel, to value. Note that this  
    is not smart enough to keep track of the different bandwidth constraints  
    on different types of waveforms, so see the manual accordingly.  
    """  
    self.write('{}:BSW FRQ, {}'.format(channel, value))
```

3. Recibir Mensajes:

El envío de mensajes se hace a través del método “read”. Este método lee el buffer hasta el punto de leer el byte de finalización¹. Este método se usa en casos donde hay un mensaje que está esperando en la cola del dispositivo. En general esto es útil después de escribir un mensaje que hace al dispositivo escribir una respuesta. Este método es usado en el método “query”, que primero hace un “write” y después lee la respuesta.

Código de “query” en la librería Lantz:

```
def query(self, command, *, send_args=(None, None), recv_args=(None, None)):
    """Send query to the instrument and return the answer

    Parameters
    -----
    command : string
        command to be sent to the instrument
    send_args :
        (termination, encoding) to override class defaults
    recv_args :
        (termination, encoding) to override class defaults
    """

    self.write(command, *send_args)
    return self.read(*recv_args)
```

Ejemplo de uso del “query” para leer una entrada:

```
@Feat(read_once=True)
def idn(self):
    """
    The *IDN? query returns the SR830's device identification string. This
    string is in the format
    "Stanford_Research_Systems,SR830,s/n00111,ver1.000".
    :return: Identification string
    """
    return self.query('*IDN?')
```

Apéndice:

Caracteres de terminación: la librería tiene un mecanismo para incluir diferentes caracteres de terminación. Por ejemplo, el generador de funciones SDG5122 usa el formato SCPI de comandos, que especifica usar el carácter “new line” como terminación. Esto se puede definir en la clase de la siguiente manera:

```
class SDG5122(MessageBasedDriver):
    """
    Class that administers and provides an interface to connect with the function generator
    SDG5122 from Siglent
    """

    DEFAULTS = {
        'COMMON': {
            'write_termination': '\n',
            'read_termination': '\n',
        }
    }
```